

Московский государственный технический университет

Факультет ИУ «Информатика и системы управления»

Кафедра ИУ-1 «Системы автоматического управления»

ОТЧЕТ

по лабораторной работе №1

«»

по дисциплине

«Микропроцессорные и встраиваемые системы»

Выполнил: Салженикин Д.А.

Галиев Э.Р.

Группа: ИУ1-52Б

Проверила: Сычев В.В.

Работа выполнена:

Отчет сдан:

Оценка:

Цель работы: Приобретение практических навыков программирования на Си и отработка в среде разработки STM32CubeIDE.

Задачи: Настройка тактирования и прерываний, настройка пинов GPIO, инициализация SPI, инициализация I2C, включение светодиодов.

Код:

```
#include "main.h"

#include "stm32l476xx.h"

extern uint32_t systemTimeCounter;
uint32_t *spi2_dr = 0x40003c00 + 0x0C;

void delayMs(uint32_t delay)
{
	uint32_t curr = systemTimeCounter;
	while(systemTimeCounter - curr < delay)
	{	}
}

void I2CTransmit(uint8_t slaveADDR, uint8_t *data, uint8_t n){
	I2C1->CR2 |= I2C_CR2_NBYTES | I2C_CR2_RELOAD;
	I2C1->CR2 |= slaveADDR << 1;
	I2C1->CR2 |= I2C_CR2_START;
	for(int i = 0; i<n;i++)
	{
		while( !(I2C1->ISR & I2C_ISR_TXIS ))
		{
			if(I2C1->ISR & I2C_ISR_NACKF)
				return;
		}
		I2C1->TXDR = data[i];
	}
}

void spiTransmitRecieve(uint8_t *txBuff, uint8_t *rxBuff,uint8_t tn, uint8_t rn)
{
	GPIOD->ODR &= ~GPIO_ODR_ODR_7;
	uint8_t dummy;
	for(int i = 0; i<tn; i++)
	{
```

```
            while(!(SPI2->SR & SPI_SR_TXE));
            SPI2->DR = txBuff[i];
            while(!(SPI2->SR & SPI_SR_RXNE));
            dummy = SPI2->DR;
        }
        while(SPI2->SR & SPI_SR_BSY);
        for(int i = 0; i<rn; i++)
            {
                while(!(SPI2->SR & SPI_SR_TXE));
                SPI2->DR = dummy;
                while(!(SPI2->SR & SPI_SR_RXNE));
                rxBuff[i] = SPI2->DR;
            }
            while(SPI2->SR & SPI_SR_BSY);
        GPIOD->ODR |= GPIO_ODR_ODR_7;

}

int main(void)
{
        // PWR AND SYS ENABLE
        RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;
        RCC->APB1ENR1 |= RCC_APB1ENR1_PWREN;
        RCC->CR |= RCC_CR_MSIRGSEL;
        SysTick_Config(4000000/1000);
        NVIC_SetPriority(SysTick_IRQn,0);
        NVIC_EnableIRQ(SysTick_IRQn);

        RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN |
RCC_AHB2ENR_GPIODEN;

    // cs
        GPIOD->MODER &= ~(3 << GPIO_MODER_MODE7_Pos);
        GPIOD->MODER |= (1 << GPIO_MODER_MODE7_Pos);
        GPIOD->OSPEEDR |= GPIO_OSPEEDER_OSPEEDR7;

        GPIOD->ODR |= GPIO_ODR_OD7;

        // SPI2 INIT
        RCC->APB1ENR1 |= RCC_APB1ENR1_SPI2EN;

        GPIOD->MODER &= ~((3 << GPIO_MODER_MODE1_Pos)|(3 <<
GPIO_MODER_MODE3_Pos)|(3 << GPIO_MODER_MODE4_Pos));
```

```
	GPIOD->MODER |= ((2 << GPIO_MODER_MODE1_Pos)|(2 << GPIO_MODER_MODE4_Pos)|(2 << GPIO_MODER_MODE3_Pos)); // 2 ALTERNATIVE

	GPIOD->OSPEEDR |= GPIO_OSPEEDER_OSPEEDR1 | GPIO_OSPEEDER_OSPEEDR3 | GPIO_OSPEEDER_OSPEEDR4;
	GPIOD->AFR[0] |=  (5 << GPIO_AFRL_AFSEL1_Pos) | (5 << GPIO_AFRL_AFSEL3_Pos) | ( 5<< GPIO_AFRL_AFSEL4_Pos);

	SPI2->CR1 &= ~SPI_CR1_BR;
	SPI2->CR1 |= (1 << SPI_CR1_BR_Pos);
	SPI2->CR1 |=  SPI_CR1_SSI| SPI_CR1_SSM;
	SPI2->CR1 |= SPI_CR1_MSTR;
	SPI2->CR2 &= ~(SPI_CR2_DS | SPI_CR2_SSOE);
	SPI2->CR2 |= (7<< SPI_CR2_DS_Pos) | SPI_CR2_SSOE;
	SPI2->CR2 |= SPI_CR2_NSSP; //( DATA SIZE BITS DONT NEDD SET? WHEN 8 BIT DATA)
//	SPI2->CR2 |= SPI_CR2_FRXTH;
	  SPI2->CR1 |= SPI_CR1_SPE;

	//I2C INIT
	RCC->CCIPR &= ~(3<< RCC_CCIPR_I2C1SEL);
	RCC->APB1ENR1 |= RCC_APB1ENR1_I2C1EN;


	GPIOB->MODER &= ~((3 << GPIO_MODER_MODE6_Pos)|(3 << GPIO_MODER_MODE7_Pos));
	GPIOB->MODER |= ((2 << GPIO_MODER_MODE6_Pos)|(2 << GPIO_MODER_MODE7_Pos));
	GPIOB->OTYPER |= GPIO_OTYPER_IDR_6 |GPIO_OTYPER_IDR_7;
	GPIOB->OSPEEDR |= GPIO_OSPEEDER_OSPEEDR6 | GPIO_OSPEEDER_OSPEEDR7;
	GPIOB->AFR[0] |=  (4 << GPIO_AFRL_AFSEL6_Pos) | (4 << GPIO_AFRL_AFSEL7_Pos);

	I2C1->TIMINGR |= (0x14 << I2C_TIMINGR_SCLL_Pos) | (0xE << I2C_TIMINGR_SCLH_Pos) ;
	I2C1->OAR1 |= I2C_OAR1_OA1EN ;
	I2C1->CR2 |= I2C_CR2_AUTOEND | I2C_CR2_NACK;
	I2C1->CR1 |= I2C_CR1_PE;

  // Led
	GPIOB->MODER &=  ~(3<< GPIO_MODER_MODE2_Pos);
```

```
    GPIOB->MODER |= 1 << GPIO_MODER_MODE2_Pos;

    GPIOB->ODR |= GPIO_ODR_OD2;


    uint8_t data[2];
     data[0] = 0x20;
     data[1] = 0x0f;
     spiTransmitRecieve(data, 0, 2, 0);
     data[0] = 0x26 | (1<<6) | (1<<7);
     uint8_t dataRecieve[8];

 while (1)
 {
      spiTransmitRecieve(data, dataRecieve, 1, 8);
    for(int i =0;i<8;i+=2)
    {
        i2cTransmit(0x10, dataRecieve+i,2);
        delayMs(1000);
    }
     delayMs(4000);
     GPIOB->ODR ^= GPIO_ODR_OD2;
 }

}
```

Пояснения к коду:

1. Настройка тактирования и прерываний:
   - Включение системного таймера SysTick с частотой 4 МГц.
   - Установка приоритета прерывания SysTick.
   - Разрешение прерывания SysTick.
2. Настройка пинов GPIO:
   - Включение тактирования для портов GPIOA и GPIOB.
   - Настройка пина для "чип-селект" на GPIOA.
   - Установка режима работы пина в режим "вывода общего назначения".
   - Настройка скорости работы пина.
   - Установка пина в режим "высокий уровень" (output).
3. Инициализация SPI:
   - Включение тактирования для SPI2.
   - Настройка пинов для SPI2 в альтернативном режиме.
   - Настройка скорости передачи данных SPI2.
   - Установка режима мастера.

- Установка размера данных SPI в 8 бит.
- Разрешение работы SPI2.

4. Инициализация I2C:
    - Установка источника тактирования для I2C.
    - Включение тактирования для I2C.
    - Настройка пинов для I2C в альтернативном режиме.
    - Настройка скорости передачи данных I2C.
    - Разрешение работы I2C.
5. Включение светодиода:
    - Настройка пина для светодиода на GPIOB.
    - Установка режима работы пина в режим "вывода общего назначения".
    - Установка пина в режим "высокий уровень" (output).
6. Основной цикл:
    - В бесконечном цикле выполняется следующее:
        - Задержка в 1000 миллисекунд с использованием функции delayMs.
        - Мигание светодиодом на GPIOB.

Вывод:

Приобрели практические навыки программирования на Си и отработали поставленные задачи в среде разработки STM32CubeIDE.